Making Sure Jobs Succeed…Eventually

We write code to solve a problem or achieve some result. When that code is split up such that some of it runs while a user is waiting and some runs in a background job, it's hard to clearly understand what code ran by looking at the server logs. For example, if your users are successfully creating orders, but none of the `CompleteOrderJob` jobs are completing, you have a problem.

The biggest challenge in managing a system with Sidekiq jobs is making sure that the jobs all eventually complete. This requires knowing what jobs have failed, why they failed, and fixing anything needed to make them succeed when retried. For even a moderately sized system, this can be an overwhelming task, at least without some sort of automation.

There are three ways to deal with this: 1) ignore them and hope for the best, 2) manually examine each job failure to figure out what to do, or 3) devise and implement a strategy to manage only those jobs that require a developer to intervene. I've been on teams that have used each one of these and, if you can't guess already, you'll be learning about the third way to deal with this.

To develop a strategy to manage failed Sidekiq jobs, you'll need to first understand why jobs fail.